

# Computer Vision Lab Manual 06: Master Cheat-Sheet

*Wavelet Transforms, Boundary Detection, Hough Transform & SIFT Feature Extraction*

## 1. Theoretical Foundations & Mathematical Formulations

### 1.1 Wavelet Transformation

Wavelet transformation is a mathematical technique used in signal processing, data analysis, and image processing. Unlike Fourier Transform (which provides frequency information without spatial localization), wavelets offer time-frequency (or spatial-frequency) multi-resolution analysis. This makes them ideal for non-stationary signals and non-uniform image features.

- **Continuous Wavelet Transform (CWT):** Involves continuous wavelet functions  $\psi(t)$  dilated by scale factor 'a' and translated by shift parameter 'b':  $CWT(a,b) = \int f(t) * (1/\sqrt{a}) * \psi((t-b)/a) dt$ . Used for oscillatory and seismic signal analysis.

- **Discrete Wavelet Transform (DWT):** Uses discrete scale and shift values (typically powers of 2). In 2D images, DWT decomposes an image into 4 subbands: LL (Approximation), LH (Horizontal Edges), HL (Vertical Edges), and HH (Diagonal Edges). Wavelet families include Daubechies, Haar, and Symlet.

- **Wavelet Packet Transform (WPT):** Extension of DWT that decomposes both approximation (LL) AND detail subbands (LH, HL, HH) recursively, allowing equal frequency resolution across high and low frequencies.

- **Inverse Wavelet Transform (IDWT):** Reconstructs the original image/signal without loss from its wavelet subband coefficients. Crucial for denoising and image compression (e.g., JPEG 2000).

### 1.2 Boundary Detection (Edge Detection)

Boundary detection isolates structural contours where sharp intensity or color changes occur. Edges reduce data volume while preserving essential geometry.

- **Sobel Operator:** 1st-derivative 3x3 convolution mask calculating horizontal ( $G_x$ ) and vertical ( $G_y$ ) gradients:  $G = \sqrt{G_x^2 + G_y^2}$ ,  $\theta = \arctan2(G_y, G_x)$ .

- **Canny Edge Detector:** Optimal multi-stage edge detector: 1) Gaussian Blur, 2) Gradient Calculation (Sobel), 3) Non-Maximum Suppression (NMS thinning), 4) Double Thresholding ( $T_{high}$ ,  $T_{low}$ ), 5) Edge Tracking by Hysteresis.

- **Laplacian of Gaussian (LoG):** 2nd-derivative operator detecting edges via zero-crossings:  $LoG = Laplacian(Gaussian(Image))$ . Highly effective for multi-scale edge detection.

### 1.3 Introduction to Hough Transformation

The Hough Transform converts spatial domain points (x, y) into parameter space accumulator voting bins to detect parametric geometric shapes (lines, circles) despite noise and occlusions.

- **Hough Line Detection:** Uses polar parametric form  $r = x \cdot \cos(\theta) + y \cdot \sin(\theta)$  to avoid infinite slope issues of  $y = mx + b$ . Edge pixels vote into an accumulator matrix  $A(r, \theta)$ . Local maxima (peaks) indicate detected lines.

- **Hough Circle Detection:** Uses parametric equation  $(x - a)^2 + (y - b)^2 = r^2$ . Parameter space is 3D (a, b, r). OpenCV uses Hough Gradient Method (2D accumulator on gradient direction) to reduce computational complexity.

### 1.4 SIFT Feature Extraction (Scale-Invariant Feature Transform)

SIFT extracts local scale-, rotation-, and illumination-invariant keypoint descriptors in 5 core stages:

- **1. Scale-Space Extrema Detection:** Searches across scale space using Difference-of-Gaussians (DoG):  $DoG(x,y,\sigma) = L(x,y,k\sigma) - L(x,y,\sigma)$ , where L is Gaussian-blurred image.

- **2. Keypoint Localization:** Compares each candidate point against 26 neighbors (8 in current scale layer, 9 in layer above, 9 in layer below). Fits 3D quadratic function to refine sub-pixel location and removes low-contrast/edge points.

- **3. Orientation Assignment:** Assigns canonical orientation based on dominant gradient orientation in local keypoint neighborhood, making descriptor rotation-invariant.

- **4. Keypoint Descriptor Generation:** Computes 16 sub-regions (4x4 grid) around keypoint. Each sub-region builds an 8-bin orientation histogram, forming a  $4 \times 4 \times 8 = 128$ -dimensional feature vector.

- **5. Keypoint Matching & Outlier Rejection:** Matches keypoints using Euclidean distance (Lowe's ratio test:  $dist1/dist2 < 0.75$ ). Outliers rejected via RANSAC before estimating 3x3 Homography.

## 2. Complete Terminology Dictionary (22 Key Terms)

- **Wavelet Transform:** Multi-resolution mathematical transform decomposing signals/images into scale-frequency subbands.
- **DWT Subbands (LL, LH, HL, HH):** LL=Approximation (low pass), LH=Horizontal details, HL=Vertical details, HH=Diagonal details.
- **Difference of Gaussians (DoG):** Scale-space operator approximating Laplacian of Gaussian by subtracting two Gaussian-blurred images:  $L(k\sigma) - L(\sigma)$ .
- **Scale Space:** Stack of image representations at increasing levels of Gaussian blur to achieve scale invariance.
- **26-Neighbor Check:** Extrema detection comparing candidate point against 8 spatial neighbors + 9 upper-scale neighbors + 9 lower-scale neighbors.
- **Canonical Orientation:** Dominant gradient orientation assigned to a keypoint to achieve rotation invariance.
- **128-D SIFT Descriptor:**  $4 \times 4$  spatial grid \* 8 orientation bins = 128-element feature vector representing local gradient structure.
- **Lowe's Ratio Test:** Matching filter keeping matches only if  $\text{best\_dist} / \text{second\_best\_dist} < 0.75$  (rejects ambiguous matches).
- **RANSAC:** Random Sample Consensus iterative algorithm that fits models and rejects outlier keypoint matches.
- **Hough Parameter Space:** Accumulator matrix where image curves map to parameter intersection points (peaks).
- **Polar Line Representation:** Line equation  $r = x \cdot \cos(\theta) + y \cdot \sin(\theta)$ , where  $r$  is perpendicular distance to origin and  $\theta$  is angle.
- **Hough Gradient Method:** OpenCV optimization for circle detection using edge gradient direction to reduce 3D accumulator to 2D.
- **Sobel Gradient Magnitude:** Edge strength  $G = \sqrt{G_x^2 + G_y^2}$  computed from 1st derivatives  $G_x$  and  $G_y$ .
- **Canny Non-Maximum Suppression:** Thinning technique keeping only local gradient intensity maxima along the gradient direction.
- **Canny Hysteresis:** Dual-threshold edge linking using  $T_{\text{high}}$  and  $T_{\text{low}}$ ; weak edges kept only if connected to strong edges.
- **Laplacian Zero-Crossings:** Locations where 2nd-derivative switches sign, indicating precise edge centers in LoG filtering.
- **Keypoint Localization:** Sub-pixel refinement using Taylor series expansion to eliminate low-contrast points and unstable edge responses.
- **Wavelet Packet Decomposition:** Recursive decomposition of both approximation and detail subbands for granular frequency analysis.
- **Continuous Wavelet Transform (CWT):** Wavelet transform using continuous scale and shift parameters for signal time-frequency analysis.
- **Homography Matrix (H):**  $3 \times 3$  projective transformation matrix mapping keypoint coordinates between two image planes.
- **PyWavelets (pywt):** Standard Python library for discrete and continuous wavelet transformations.
- **BFMatcher (Brute-Force Matcher):** OpenCV descriptor matcher comparing Euclidean distance across all keypoint pairs.

### 3. Exhaustive Function & Parameter Breakdown Reference

Detailed reference for all PyWavelets and OpenCV functions used in Lab Manual 06 pipelines.

#### Function 1: `pywt.dwt2(data, wavelet, mode='symmetric')`

Computes 2D Discrete Wavelet Transform decomposing an image into 4 subbands.

| Parameter | Type / Range              | Description & Mathematical Role                                                     |
|-----------|---------------------------|-------------------------------------------------------------------------------------|
| data      | ndarray (2D)              | Input 2D grayscale image array.                                                     |
| wavelet   | str or Wavelet            | Wavelet family name (e.g., 'haar', 'db1', 'db4', 'sym2').                           |
| mode      | str (default='symmetric') | Signal extension mode for boundary handling ('symmetric', 'zero', 'periodization'). |

Return Value: Tuple (cA, (cH, cV, cD)) where cA=LL (Approximation), cH=LH (Horizontal), cV=HL (Vertical), cD=HH (Diagonal).

#### Function 2: `cv2.HoughLines(image, rho, theta, threshold)`

Standard Hough Transform for line detection in binary edge images.

| Parameter | Type / Range    | Description & Mathematical Role                                             |
|-----------|-----------------|-----------------------------------------------------------------------------|
| image     | ndarray (uint8) | Input 8-bit single-channel binary edge map (e.g., Canny output).            |
| rho       | float (>0)      | Distance resolution of accumulator in pixels (typically 1.0).               |
| theta     | float (>0)      | Angle resolution of accumulator in radians (typically $\text{np.pi}/180$ ). |
| threshold | int (>0)        | Accumulator threshold. Only lines receiving votes > threshold are returned. |

Return Value: Array of shape (N, 1, 2) containing line parameters (rho, theta).

#### Function 3: `cv2.HoughLinesP(image, rho, theta, threshold, minLineLength, maxLineGap)`

Probabilistic Hough Transform returning line segment end-point coordinates (x1, y1, x2, y2).

| Parameter     | Type / Range      | Description & Mathematical Role                               |
|---------------|-------------------|---------------------------------------------------------------|
| image         | ndarray (uint8)   | Input 8-bit single-channel binary edge map.                   |
| rho           | float             | Distance resolution of accumulator in pixels (e.g., 1.0).     |
| theta         | float             | Angle resolution in radians (e.g., $\text{np.pi}/180$ ).      |
| threshold     | int               | Minimum accumulator votes required to accept a line segment.  |
| minLineLength | float (default=0) | Minimum line segment length. Shorter segments discarded.      |
| maxLineGap    | float (default=0) | Maximum allowed gap between points on same line to link them. |

Return Value: Array of shape (N, 1, 4) containing line endpoints  $[[x_1, y_1, x_2, y_2]]$ .

#### Function 4: `cv2.HoughCircles(image, method, dp, minDist, param1, param2, minRadius, maxRadius)`

Detects circular shapes in grayscale image using 2D Hough Gradient method.

| Parameter | Type / Range    | Description & Mathematical Role                                                                |
|-----------|-----------------|------------------------------------------------------------------------------------------------|
| image     | ndarray (uint8) | Input 8-bit single-channel grayscale image.                                                    |
| method    | OpenCV Flag     | Detection method. Must be <code>cv2.HOUGH_GRADIENT</code> or <code>HOUGH_GRADIENT_ALT</code> . |
| dp        | float (>=1)     | Inverse ratio of accumulator resolution to image resolution (dp=1 -> same size).               |
| minDist   | float (>0)      | Minimum distance between centers of detected circles (prevents duplicate detections).          |
| param1    | float           | First method-specific parameter. Upper threshold for Canny edge detector.                      |
| param2    | float           | Second parameter. Accumulator threshold for circle centers (smaller -> more false circles).    |
| minRadius | int (>=0)       | Minimum circle radius in pixels.                                                               |
| maxRadius | int (>=0)       | Maximum circle radius in pixels.                                                               |

Return Value: Array of shape (1, N, 3) containing  $[[x\_center, y\_center, radius]]$ .

#### Function 5: `cv2.SIFT_create(nfeatures, nOctaveLayers, contrastThreshold, edgeThreshold, sigma)`

Instantiates SIFT detector and descriptor extractor object.

| Parameter         | Type / Range         | Description & Mathematical Role                                                |
|-------------------|----------------------|--------------------------------------------------------------------------------|
| nfeatures         | int (default=0)      | Number of best keypoints to retain (0 = retain all).                           |
| nOctaveLayers     | int (default=3)      | Number of layers in each octave of scale space.                                |
| contrastThreshold | float (default=0.04) | Threshold to filter out weak keypoints in low-contrast regions.                |
| edgeThreshold     | float (default=10)   | Threshold to filter out edge-like keypoints (Hessian matrix eigenvalue ratio). |
| sigma             | float (default=1.6)  | Gaussian blur sigma for the base scale space layer.                            |

#### Function 6: `sift.detectAndCompute(image, mask)`

Detects keypoints and computes 128-D SIFT descriptors in a single step.

| Parameter | Type / Range    | Description & Mathematical Role                                                   |
|-----------|-----------------|-----------------------------------------------------------------------------------|
| image     | ndarray (uint8) | Input 8-bit single-channel grayscale image.                                       |
| mask      | ndarray or None | Optional mask specifying where to search for keypoints. Pass None for full image. |

Return Value: Tuple (keypoints, descriptors) where keypoints is list of `cv2.KeyPoint` objects and descriptors is float32 array of shape (N, 128).

## 4. Fully Annotated Code Implementations

### Program 1: 2D Discrete Wavelet Transform (DWT Decomposition & Reconstruction)

```
import pywt
import numpy as np
import cv2
import matplotlib.pyplot as plt

# 1. Load grayscale image
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# 2. Perform 2D Discrete Wavelet Transform using Haar wavelet
# Returns LL (Approximation), (LH, HL, HH) (Horizontal, Vertical, Diagonal details)
coeffs2 = pywt.dwt2(image, 'haar')
LL, (LH, HL, HH) = coeffs2

# 3. Perform Inverse 2D DWT to reconstruct original image
reconstructed = pywt.idwt2(coeffs2, 'haar')

# Plot results
fig, axes = plt.subplots(2, 2, figsize=(8, 8))
axes[0, 0].imshow(LL, cmap='gray'); axes[0, 0].set_title('LL (Approximation)')
axes[0, 1].imshow(LH, cmap='gray'); axes[0, 1].set_title('LH (Horizontal Edges)')
axes[1, 0].imshow(HL, cmap='gray'); axes[1, 0].set_title('HL (Vertical Edges)')
axes[1, 1].imshow(HH, cmap='gray'); axes[1, 1].set_title('HH (Diagonal Edges)')
plt.tight_layout(); plt.show()
```

### Program 2: Boundary Detection Suite (Sobel, Canny & Laplacian of Gaussian)

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load image and convert to grayscale
image = cv2.imread('image.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# A. SOBEL EDGE DETECTION
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel_mag = np.sqrt(sobel_x**2 + sobel_y**2)
sobel_mag = np.uint8(np.clip(sobel_mag, 0, 255))

# B. CANNY OPTIMAL EDGE DETECTION
blurred = cv2.GaussianBlur(gray, (5, 5), 1.4)
canny_edges = cv2.Canny(blurred, threshold1=50, threshold2=150)

# C. LAPLACIAN OF GAUSSIAN (LoG) EDGE DETECTION
log_laplacian = cv2.Laplacian(blurred, cv2.CV_64F)
log_abs = cv2.convertScaleAbs(log_laplacian)
```

### Program 3: Line & Circle Hough Transformation

```
import cv2
import numpy as np

# 1. Probabilistic Line Hough Transform
image = cv2.imread('road.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)

lines = cv2.HoughLinesP(edges, rho=1, theta=np.pi/180, threshold=50, minLineLength=50, maxLineGap=10)
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0]
        cv2.line(image, (x1, y1), (x2, y2), (0, 255, 0), 2)

# 2. Circle Hough Transform
coins = cv2.imread('coins.jpg', cv2.IMREAD_GRAYSCALE)
coins_blur = cv2.medianBlur(coins, 5)
circles = cv2.HoughCircles(coins_blur, cv2.HOUGH_GRADIENT, dp=1, minDist=20, param1=50, param2=30, minRadius=10, maxRadius=50)

if circles is not None:
```

```
circles = np.uint16(np.around(circles))
for i in circles[0, :]:
    cv2.circle(coins, (i[0], i[1]), i[2], (0, 255, 0), 2) # Circle boundary
    cv2.circle(coins, (i[0], i[1]), 2, (0, 0, 255), 3)    # Circle center
```

#### Program 4: SIFT Feature Extraction, Matching & RANSAC Homography

```
import cv2
import numpy as np

img1 = cv2.imread('query.jpg', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('scene.jpg', cv2.IMREAD_GRAYSCALE)

# 1. Initialize SIFT Detector
sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)

# 2. Brute-Force Matcher with KNN (k=2)
bf = cv2.BFMatcher(cv2.NORM_L2)
matches = bf.knnMatch(des1, des2, k=2)

# 3. Apply Lowe's Ratio Test
good_matches = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

# 4. Extract location of good matches & estimate Homography via RANSAC
if len(good_matches) > 10:
    src_pts = np.float32([kp1[m.queryIdx].pt for m in good_matches]).reshape(-1, 1, 2)
    dst_pts = np.float32([kp2[m.trainIdx].pt for m in good_matches]).reshape(-1, 1, 2)

    H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
    print("Estimated 3x3 Homography Matrix:\n", H)
```

## 5. Quick Reference & Exam Survival Kit

### 5.1 Master Comparison Matrix

| Method / Technique | Input / Domain       | Key Strengths                                | Primary Weakness / Limitation                 |
|--------------------|----------------------|----------------------------------------------|-----------------------------------------------|
| DWT Wavelets       | 2D Image / Frequency | Multi-resolution spatial-frequency analysis. | Computationally heavier than Fourier.         |
| Sobel Edge         | Grayscale Image      | Fast 1st-derivative gradient operator.       | Sensitive to high-frequency noise.            |
| Canny Edge         | Grayscale Image      | Optimal thinning (NMS) & hysteresis linking. | Requires tuning 2 thresholds.                 |
| LoG Operator       | Grayscale Image      | Accurate edge centers via zero-crossings.    | Prone to ringing / false edges at corners.    |
| Hough Line         | Binary Edge Map      | Detects global lines despite gaps/noise.     | High accumulator memory & computation.        |
| Hough Circle       | Grayscale Image      | Detects circular objects (coins/cells).      | Sensitive to param1/param2 center thresholds. |
| SIFT Extrema       | Scale Space (DoG)    | Scale invariant keypoint detection.          | More complex than Harris corners.             |
| SIFT Descriptor    | Keypoint Patch       | 128-D rotation & illumination invariant.     | Computationally slower than ORB.              |

### 5.2 High-Yield MCQ Traps & Exam Trick Alerts

- **Trap 1: SIFT Descriptor Dimensionality:** SIFT descriptor uses a 4x4 grid of sub-regions, each with an 8-bin orientation histogram:  $4 \times 4 \times 8 = \text{EXACTLY } 128 \text{ NUMBERS!}$
- **Trap 2: DoG Scale Space Neighbors:** To confirm a keypoint as an extremum, SIFT compares its DoG value against 26 NEIGHBORS (8 in current scale + 9 in scale above + 9 in scale below).
- **Trap 3: Lowe's Ratio Test Threshold:** Lowe's ratio test checks  $\text{best\_dist} < 0.75 * \text{second\_best\_dist}$ . Rejecting ratios  $> 0.75$  eliminates ambiguous background matches.
- **Trap 4: Hough Lines Accumulator Parameter Space:** Standard Hough lines use polar form (rho, theta) NOT slope-intercept (m, b) to prevent infinite slope division errors on vertical lines!
- **Trap 5: Sobel Negative Gradients:** cv2.Sobel requires cv2.CV\_64F output depth! Saving directly as uint8 clips negative gradient values, destroying half the edge directions.

### 5.3 'Which Function Do I Use?' Decision Matrix

| Exam Task Requirement                     | Primary Function / API | Key Flags / Parameters                                 |
|-------------------------------------------|------------------------|--------------------------------------------------------|
| Multi-resolution subband decomposition    | pywt.dwt2()            | data, 'haar' or 'db1'                                  |
| Detect thin continuous optimal edges      | cv2.Canny()            | image, threshold1, threshold2                          |
| Detect edges via zero-crossings           | cv2.Laplacian()        | GaussianBlur image, cv2.CV_64F                         |
| Detect line segments with endpoints       | cv2.HoughLinesP()      | edges, rho=1, theta=pi/180, thresh, minLineLength      |
| Detect circular objects / coins           | cv2.HoughCircles()     | image, cv2.HOUGH_GRADIENT, dp, minDist, param1, param2 |
| Extract scale/rotation invariant features | cv2.SIFT_create()      | sift.detectAndCompute(gray, None)                      |
| Match SIFT descriptors robustly           | cv2.BFMatcher()        | bf.knnMatch(des1, des2, k=2) + Lowe's ratio test       |
| Reject outlier matches & estimate warp    | cv2.findHomography()   | src_pts, dst_pts, cv2.RANSAC, 5.0                      |

### 5.4 Common Coding Errors & Fixes

- **1. SIFT Descriptor Creation Error in OpenCV:** In older OpenCV versions, SIFT was in xfeatures2d (cv2.xfeatures2d.SIFT\_create()). In modern OpenCV (4.4+), it is cv2.SIFT\_create().
- **2. HoughCircles Returns None / Crashes:** HoughCircles requires a median/Gaussian blurred grayscale image. If no circles match param1/param2, it returns `None` - always check `if circles is not None:` before looping!
- **3. Reshaping Keypoints for Homography:** findHomography requires point arrays of shape `(N, 1, 2)` and data type `float32`. Use `np.float32([...]).reshape(-1, 1, 2)`.

### 5.5 Last-Minute Revision Checklist

- **DWT Subbands:** LL = Low-Low (Approximation), LH = Low-High (Horizontal), HL = High-Low (Vertical), HH = High-High (Diagonal).
- **Canny 5 Stages:** Gaussian Blur -> Sobel Gradient -> Non-Max Suppression (Thins edges) -> Double Threshold -> Hysteresis Tracking.
- **Hough Space Line:**  $r = x \cdot \cos(\theta) + y \cdot \sin(\theta)$ . Intersection point in (r, theta) space = line in image space.
- **SIFT 5 Steps:** 1) DoG Scale Extrema, 2) Keypoint Localization (26-neighbor check), 3) Orientation Assignment, 4) 128-D Descriptor, 5) Matching & RANSAC.